

Labo 01 — Labo pratique : observer l'isolation de ses propres yeux

Objectif : vérifier expérimentalement chaque affirmation de la théorie. À la fin, vous aurez vu de vos yeux qu'un conteneur est un processus de votre hôte.

Prérequis — Docker Engine installé sur Linux, `docker version` renvoie un bloc *Client* **et** un bloc *Server*. Aucun fichier n'est nécessaire pour ce labo.

Étape 1 — Identifier les deux moitiés de Docker

```
docker version
```

Observez deux blocs distincts, `Client:` et `Server: Docker Engine - Community`, avec chacun sa version. Ils peuvent différer : un client 29.x parle sans problème à un daemon 28.x.

```
docker info | head -n 25
```

Observez les lignes `Containers:`, `Images:`, `Storage Driver: overlay2`, `Cgroup Driver: systemd`, `Cgroup Version: 2`, `Kernel Version:`.

Explication. `docker version` interroge les deux moitiés ; `docker info` décrit l'état du **daemon** uniquement. Le `Storage Driver` est le composant qui empile les couches d'image, et le `Cgroup Version` celui qui appliquera vos limites de ressources.

Étape 2 — Le premier conteneur, et où il est passé

```
docker run alpine echo "bonjour depuis le conteneur"
```

Observez le téléchargement de l'image (`Unable to find image locally` puis `Pull complete`), le message affiché... puis le retour immédiat au prompt.

```
docker ps
docker ps -a
```

Observez que `docker ps` ne montre **rien**, mais que `docker ps -a` montre le conteneur avec le statut `Exited (0)`.

Explication. Un conteneur vit exactement le temps de son processus principal. `echo` a écrit une ligne puis s'est terminé : le conteneur est mort avec lui, mais il n'est pas supprimé pour autant — il reste comme un cadavre inspectable. `docker ps` ne liste que les conteneurs en cours d'exécution.

```
docker run --rm alpine echo "celui-ci ne laissera pas de trace"
docker ps -a
```

Observez qu'aucun nouveau conteneur n'apparaît : `--rm` supprime automatiquement le conteneur à sa sortie.

Étape 3 — Le noyau est celui de l'hôte

```
uname -r
docker run --rm alpine uname -r
docker run --rm debian uname -r
```

Observez que les **trois** commandes affichent exactement la même version de noyau, alors que l'hôte, Alpine et Debian sont trois systèmes différents.

```
docker run --rm alpine cat /etc/os-release | head -n 2
docker run --rm debian cat /etc/os-release | head -n 2
```

Observez cette fois deux résultats différents : `Alpine Linux` et `Debian GNU/Linux`.

Explication. La preuve est faite : l'image apporte le *userland* (fichiers, binaires, bibliothèques), le noyau vient de l'hôte et n'est jamais dupliqué.

Étape 4 — Voir le processus depuis les deux côtés

Lancez un conteneur qui dure :

```
docker run -d --name veilleur alpine sleep 600
docker ps
```

Observez le statut `Up`, le nom `veilleur`, et l'`IMAGE` utilisée.

Vue **depuis l'intérieur** :

```
docker exec veilleur ps -o pid,ppid,comm
```

Observez une liste minuscule : `sleep` porte le **PID 1**.

Vue **depuis l'hôte** :

```
ps -ef | grep "[s]leep 600"
docker inspect --format '{{.State.Pid}}' veilleur
```

Observez que le même processus existe sur l'hôte, avec un PID tout à fait ordinaire (par exemple 48213), et que `docker inspect` vous donne précisément ce PID.

Explication. Un seul et même processus, deux numérotations. À l'intérieur, le namespace `pid` lui fait croire qu'il est le premier processus du système ; à l'extérieur, il n'est qu'un processus parmi des centaines. C'est toute l'idée du conteneur.

Vérifiez maintenant qu'on peut retirer cette isolation :

```
docker run --rm --pid=host alpine ps -o pid,comm | head -n 10
```

Observez les processus de **votre hôte** (`systemd`, `dockerd` ...) listés depuis l'intérieur d'un conteneur.

Explication. L'isolation est une option, pas une propriété intrinsèque. C'est pourquoi `--pid=host` et `--privileged` sont interdits par défaut en production.

Étape 5 — Image immuable, conteneur jetable

```
docker run -d --name c1 alpine sleep 600
docker run -d --name c2 alpine sleep 600
docker exec c1 sh -c 'echo "donnee de c1" > /marque.txt'
```

Vérifiez l'isolation des écritures :

```
docker exec c1 cat /marque.txt      # affiche : donnee de c1
docker exec c2 cat /marque.txt      # erreur : No such file or directory
```

Vérifiez que l'image, elle, n'a pas bougé :

```
docker run --rm alpine ls /marque.txt  # No such file or directory
```

Observez que le fichier n'existe que dans la couche d'écriture de `c1`.

Mesurez cette couche :

```
docker ps -s --format 'table {{.Names}}\t{{.Size}}'
```

Observez une taille du type `8.19kB (virtual 9.1MB)` pour `c1` contre `4.1kB (virtual 9.1MB)` pour `c2` : le `virtual` est la taille image + couche, la première valeur est ce que le conteneur consomme **en propre**. `c1` pèse plus lourd : c'est votre fichier.

Enfin, détruisez et recommencez :

```
docker rm -f c1
docker run -d --name c1 alpine sleep 600
docker exec c1 ls /marque.txt      # No such file or directory
```

Explication. `docker rm` détruit le conteneur **et** sa couche d'écriture. Le nouveau `c1` repart de l'état exact de l'image. Toute donnée à conserver doit sortir du conteneur : c'est l'objet du labo 06.

Étape 6 — Les cgroups, ou la limite de consommation

```
docker run -d --name limite --memory=128m --memory-swap=128m alpine sleep 600
docker stats --no-stream limite
```

Observez la colonne `MEM USAGE / LIMIT` : `... / 128MiB`, et non la RAM totale de votre machine.

Comparez avec un conteneur sans limite :

```
docker stats --no-stream veilleur
```

Observez que la limite affichée est la RAM totale de l'hôte.

Explication. Sans `--memory`, un conteneur peut consommer toute la mémoire de la machine. Le namespace ne protège de rien ici : c'est le cgroup qui plafonne.

Étape 7 — inspect, la source de vérité

```
docker inspect veilleur | head -n 30
```

C'est verbeux : ciblez ce qui vous intéresse avec un *format Go*.

```
docker inspect --format '{{.State.Status}}' veilleur
docker inspect --format '{{.Config.Image}}' veilleur
docker inspect --format '{{json .Config.Cmd}}' veilleur
docker inspect --format '{{.NetworkSettings.Networks.bridge.IPAddress}}' veilleur
```

Observez respectivement `running`, `alpine`, `["sleep","600"]`, et une IP privée du type `172.17.0.2`.

Piège — beaucoup de tutoriels écrivent `{{.NetworkSettings.IPAddress}}`. Ce champ de premier niveau a disparu des versions récentes de Docker : la commande échoue avec `map has no entry for key "IPAddress"`. L'information vit désormais **par réseau**, sous `.NetworkSettings.Networks.<nom_du_réseau>`. En cas de doute, affichez d'abord la structure brute : `docker inspect --format '{{json .NetworkSettings}}' veilleur`.

Explication. `docker inspect` fonctionne sur **tous** les objets (conteneur, image, volume, réseau) et donne l'état réel, sans mise en forme. C'est l'outil de diagnostic numéro un : quand la documentation et la réalité divergent, `inspect` a raison.

Comparez avec les métadonnées de l'**image** :

```
docker image inspect --format '{{json .Config.Cmd}}' alpine
docker image inspect --format '{{.Architecture}}/{{.Os}}' alpine
```

Observez que l'image porte elle aussi une commande par défaut (`["/bin/sh"]`), que votre `sleep 600` a écrasée au `run`.

Étape 8 — La CLI, forme longue et forme courte

Vérifiez que les deux écritures sont interchangeables :

```
docker container ls -a
docker ps -a

docker image ls
docker images
```

Observez des sorties identiques.

```
docker container ls --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
```

Explication. `--format` accepte un gabarit Go et rend les sorties exploitables en script — bien plus fiable que de découper le tableau par défaut avec `awk`.

Nettoyage

```
docker rm -f veilleur c1 c2 limite
docker ps -a
```

Il reste les conteneurs `Exited` de l'étape 2. Repérez leur identifiant dans la colonne `CONTAINER ID` et supprimez-les nommément :

```
docker ps -a --filter ancestor=alpine --filter status=exited
docker rm <ID1> <ID2>
```

Et si vous voulez récupérer l'espace de l'image Debian, qui ne resservira pas :

```
docker images
docker rmi debian          # on garde alpine pour les labos suivants
```

Attention — vous croiserez partout `docker container prune`, `docker image prune -a` et `docker system prune -a`. Ces commandes ne suppriment pas « ce que vous venez de faire » mais **tout ce qui n'est pas utilisé sur la machine** : les images et conteneurs d'autres projets partent avec. Sur un poste de travail où tournent d'autres stacks, ou sur un serveur partagé, supprimez toujours nommément. Nous verrons `prune` proprement au labo 10.

Ce que vous devez pouvoir affirmer maintenant

- Le noyau affiché dans un conteneur est celui de l'hôte — vous l'avez vérifié.
- Le processus d'un conteneur existe dans le `ps` de l'hôte — vous l'avez vu, avec son PID.
- Une écriture dans un conteneur n'atteint ni l'image, ni les autres conteneurs.
- `docker rm` détruit les données ; `docker stop` non.
- Sans `--memory`, aucune limite de RAM ne s'applique.
- `docker inspect --format` est votre premier réflexe de diagnostic.